מבחן בית – מפתח/ת AI (RAG)

רקע

אופק ביטוח היא חברת ביטוח פיקטיבית שמוכרת ביטוח רכב, דירה ונסיעות לחו"ל. נציגי המוקד שלה מבזבזים זמן רב על חיפוש תשובות בעשרות מסמכים: פוליסות, נספחים, נהלים פנימיים וקטלוג טפסים.

החברה רוצה עוזר AI לנציגים. הנציג שואל שאלה בשפה חופשית, ומקבל תשובה מדויקת עם הפניה למקור. תשובה שגויה עלולה לעלות לחברה כסף או לפגוע בלקוח, ולכן עדיף שהעוזר יגיד "לא מצאתי" מאשר שימציא.

המסמכים אמיתיים במובן אחד חשוב: הם מבולגנים כמו מסמכים בארגון אמיתי.

המשימה

לבנות מערכת RAG מקצה לקצה שעונה על שאלות נציגים מתוך הקורפוס המצורף, ולהוכיח במספרים כמה היא טובה.

אנחנו לא מחפשים את ה-pipeline המתוחכם ביותר. אנחנו מחפשים מערכת שעובדת, החלטות מנומקות, ויכולת לומר בדיוק איפה המערכת טועה ולמה.

מה מצורף

קובץ zip אחד, `ofek_rag_assignment.zip`, עם שתי תיקיות:

תיקייה	תוכן
`corpus/`	33 מסמכים של אופק ביטוח בפורמטים Markdown, PDF ו-Word, ברובם בעברית וחלקם באנגלית
`dev_set/`	`dev_questions.json`: 15 שאלות לדוגמה, לכל אחת תשובת רפרנס ומסמכי המקור

השאלות לדוגמה מייצגות את סוגי השאלות שנציגים שואלים. בשיחת ההמשך נריץ על המערכת סט נוסף של שאלות שלא ראיתם.

דרישות חובה

1. **Ingestion**: קריאה של כל שלושת הפורמטים, כולל טבלאות שבתוך מסמכי PDF ו-Word.

2. **Retrieval**: שליפת הקטעים הרלוונטיים לשאלה. בחירת הגישה (vector, hybrid, reranking וכו') היא שלכם, ובלבד שתנמקו אותה.
3. **Generation**: תשובה בעברית, מבוססת על הקטעים שנשלפו בלבד.
4. **ציטוט מקורות**: כל תשובה מציינת את המסמך (ורצוי גם את הסעיף) שעליו היא נשענת.
5. **ידיעת גבולות**: כשהתשובה לא נמצאת בקורפוס, המערכת אומרת זאת ולא ממציאה.
6. **Evaluation**: סקריפט שמריץ את סט השאלות לדוגמה ומודד בנפרד את איכות השליפה ואת איכות התשובה.
7. **ממשק**: API או ממשק צ'אט פשוט, שמקבל שאלה ומחזיר תשובה עם מקורות.

מה מגישים

ריפו ב-GitHub (ציבורי או עם הרשאה אלינו) שכולל:

- **קוד שרץ** בפקודה אחת או שתיים, עם הוראות התקנה. אם צריך מפתחות API, ציינו אילו.
- **תוצאות ה-eval** על סט השאלות לדוגמה, כולל פירוט לכל שאלה ולא רק ממוצע.
- **README** שמסביר את החלטות התכנון: איך חילקתם את המסמכים, איזה embedding ולמה, איך עובדת השליפה, איך המערכת מחליטה שאין תשובה, ומה העלות וזמן התגובה המשוערים לשאלה.
- **ניתוח כשלים**: אילו שאלות נכשלו ומה לדעתכם הסיבה.
- **"עם עוד שבוע"**: מה הייתם משפרים, לפי סדר עדיפות, ולמה.

כללים

- **זמן**: המשימה מתוכננת ל-6–8 שעות עבודה. יש לכם 5 ימים להגיש. אין צורך להשקיע יותר; עדיף פתרון ממוקד עם ניתוח טוב מאשר פתרון רחב ושטחי.
- **כלי AI**: מותר ואף רצוי להשתמש ב-Claude, Cursor, Copilot וכו'. כך עובדים אצלנו. בקשה אחת: כתבו ב-README באילו כלים השתמשתם ולמה. בשיחת ההמשך תצטרכו להסביר כל שורה.
- **שאלות**: אפשר לשאול אותנו במייל. גם החלטה שקיבלתם בעצמכם בלי לשאול היא תשובה לגיטימית, כל עוד היא מתועדת.

אחרי ההגשה

נקבע שיחה של 60 דקות שבה:

1. תציגו את המערכת ואת החלטות התכנון (10 דקות).
2. נבקש שינוי קטן בקוד, בזמן אמת (15 דקות).
3. שאלות פתוחות על RAG בפרודקשן (10 דקות).

הציון לא נקבע לפי אחוז הדיוק בלבד. מערכת שמגיעה לדיוק בינוני ואתם יודעים בדיוק למה, שווה יותר ממערכת מדויקת שאינכם יודעים להסביר.